

Research Code Documentation

Dan Schroeder

January 2025

1 Hamiltonian Generation and Description of Systems

1. `ising_model.hamiltonian(Lenth, J, g, periodic)`
 - Generates the Hamiltonian according to the nearest neighbor transverse field ising model.

Parameters:

- `Length`
 - This sets the number of spins in the system
- `J`
 - This sets the multiplicative energy scale constant multiplying the entire Hamiltonian
- `g`
 - This sets the strength of the magnetic field. It is the multiplicative constant multiplying the magnetic field term
- `periodic`
 - This can only be either 0 or 1. If 0 it generates a Hamiltonian without periodic boundary conditions. If 1 it generates a Hamiltonian with periodic boundary conditions.

Returns:

- `H`
 - The resulting Hamiltonian as a quantum object.
2. `varying_range_hamiltonian(length, jdef, alpha, h)`
 - Generates the Hamiltonian according to either an infinite range model or a longer range model with a decaying interaction strength.

Parameters:

- Length
 - This sets the number of spins in the system
- jdef
 - This determines whether the function will generate according to an infinite or long range model. For an infinite range model, set jdef = 0, and for a long range model, set jdef = 1
- alpha
 - This sets the decay strength of the interactions for the long range model.
- h
 - This sets the strength of the transverse field.

Returns:

- H
 - The resulting Hamiltonian as a quantum object.
3. simplified_groundstate(length, states)
– This function will take the classical ground states of a system (which are given as binary strings) and turn these into vectors which are quantum objects. These ground state vectors can then be used to create an equal superposition of classical ground states.

Parameters:

- length
 - This is the number of spins in the system.
- states
 - This is a list of the classical ground states where each element in the list is a binary string with as many numbers as there are spins in the system. 0 corresponds to an up spin and 1 corresponds to a down spin. This list is created by the find_degeneracy function.

Returns:

- statevecs
 - A list containing the ground state vectors of the system as quantum objects
4. generate_jij(length, num_ones, seed)
– This will randomly generate the jij values which govern the spin interactions for a given instance. There are $N * (N - 1) / 2$ jij values. Typically half are positive one and half are negative one.

Parameters:

- `length`
 - This is the number of spins in the system.
- `num_ones`
 - This is how many positive ones you would like in the ultimate list of `jij` values. Typically the list is half positive one and half negative one
- `seed`
 - This is the seed numpy will use to generate the list. Each seed will generate an instance compared to each other seed, but the same seed will always result in the same list of `jij` values.

Returns:

- `jij`
 - An array containing the diagonal elements of the `jij` matrix governing the interactions. This list is used to create the interaction half of the spin glass Hamiltonian. It is part of what defines a particular instance.

5. `spin_glass_hamiltonian(length, jij, h)`
 - This will generate a Hamiltonian for the transverse field ising spin glass model according to the instance dictated by the number of spins, `jij` array, and field strength. This function will generate the interaction term and field term simultaneously. Often it is faster to calculate these separately, as in the `sg_interaction` and `sg_field` functions.

Parameters:

- `length`
 - This is the number of spins in the system.
- `jij`
 - An array containing the diagonal elements of the `jij` matrix governing the interactions. This list is used to create the interaction half of the spin glass Hamiltonian. It is part of what defines a particular instance.
- `h`
 - This is a multiplicative constant for the field term. It governs the strength of the transverse field.

Returns:

- `H`
 - The hamiltonian according to the specified instance. It is returned as a quantum object.

- Hintdiag
 - The diagonal of the interaction term. This is important for analysis of classical solutions and is used in multiple other functions.
6. `sg_interaction(length, jij)`
- This will generate the interaction term of the spin glass Hamiltonian dictated by the number of spins and the array of interaction terms. At this point it is entirely classical.

Parameters:

- length
 - This is the number of spins in the system.
- jij
 - An array containing the diagonal elements of the jij matrix governing the interactions. This list is used to create the interaction half of the spin glass Hamiltonian. It is part of what defines a particular instance.

Returns:

- Hint
 - The interaction term of the hamiltonian according to the specified instance. It is returned as a quantum object.
 - Hintdiag
 - The diagonal of the interaction term. This is important for analysis of classical solutions and is used in multiple other functions.
7. `sg_field(length)`
- This will generate the field term of the spin glass Hamiltonian dictated by the number of spins. It is returned as a quantum object and is at this point it is a generic field term for any instance with the specified number of spins. It must be multiplied by h , the field strength when constructing the final Hamiltonian.

Parameters:

- length
 - This is the number of spins in the system.

Returns:

- Hfield
 - The field term of the hamiltonian according to the specified number of spins. It is returned as a quantum object.

2 Analysis

1. `mutualinfo(length, psi, A, B)`
 - This function finds the mutual information for a single pair of spins in a given system as well as the corresponding mutual information distance.

Parameters:

- `length`
 - The number of spins in the system
- `psi`
 - The ground state of the system, given as a quantum object.
- `A`
 - The first spin of the pair. It must be passed into the function in the form `[A]`
- `B`
 - The second spin of the pair. It must be passed into the function in the form `[B]`

Returns:

- `mutualinfo`
 - The mutual information for the given pair of spins. The mutual information is not normalized.
 - `distance`
 - The mutual information distance between the pair of spins. This distance is normalized by division of the maximum value $2 \log(2)$
2. `correlation_function(length, psi, zvar)`
 - This function will calculate the value of the correlation function for every possible, non-repeating pair of spins in a system as well as the corresponding correlation function distances.

Parameters:

- `length`
 - The number of spins in the system
- `psi`
 - The ground state of the system, given as a quantum object.
- `zvar`
 - This determines whether the z or x correlation function is used. It can only be 0 or 1. If 0 it calculates the z correlation function, and if 1 it calculates the x correlation function.

Returns:

- correlationfuncvals
 - The correlation function value for every possible pair of spins in the system. it is indexed to the first value corresponds to spin pair [0,1], the second value is [0,2], and so on.
 - distances
 - The set of distances calculated from the correlation function values. It follows the same indexing convention as does the list of correlation function values. The same index in each list refers to the same pair of spins.
3. correlation_function_one_pos(length, pos, psi, zvar)
- This function will calculate the value of the correlation function for every possible, non-repeating pair of spins with respect to one specific, specified position in the spin chain as well as the corresponding correlation function distances.

Parameters:

- length
 - The number of spins in the system
- pos
 - the position in the spin chain with which to make the possible combinations
- psi
 - The ground state of the system, given as a quantum object.
- zvar
 - This determines whether the z or x correlation function is used. It can only be 0 or 1. If 0 it calculates the z correlation function, and if 1 it calculates the x correlation function.

Returns:

- correlationfuncvals
 - The correlation function value for every possible pair of spins in the system. it is indexed to the first value corresponds to spin pair [pos,0], the second value is [pos,2], and so on, where pos is the indicated spin in the parameters.
- distances
 - The set of distances calculated from the correlation function values. It follows the same indexing convention as does the list of correlation function values. The same index in each list refers to the same pair of spins.

4. `excited_states(eigenenergies, minenergy, degeneracy)`
– This function will return the first N excited states as well as the energy difference to the ground state, where N is the classical degeneracy of the system.

Parameters:

- `eigenenergies`
– A list of eigenenergies for the system in question.
- `minenergy`
– The ground state energy for the system in question.
- `degeneracy`
– The degeneracy of the ground state for the system in question

Returns:

- `excitedstates`
– The first N excited state energies of the system where N is the degeneracy of the system.
- `energy_diff`
– The energy difference from the first N excited state energies to the ground state, where N is the degeneracy of the system

5. `find_degeneracy(length, Hintdiag)`
– This function will find the classical degeneracy of the system, as well as giving the ground state configurations as binary strings.

Parameters:

- `length`
– This is the number of spins in the system
- `Hintdiag`
– This is the diagonal of the interaction term of the spin glass Hamiltonian. This is given as a return for the functions which are used to generate the spin glass Hamiltonians.

Returns:

- `degeneracy`
– The classical degeneracy of the system
- `states`
– This is a list of the classical ground states where each element in the list is a string of binary with as many numbers as there are spins in the system. 0 corresponds to an up spin and 1 corresponds to a down spin.

6. `groundstate_prob(length, jij, hlist)`
- This function calculates the probability of each ground state over a given set of h values (magnetic field strengths). In its current state, this function will only work for a system with four ground states. It can be easily modified to include more or fewer.

Parameters:

- `length`
 - The number of spins in the system
- `jij`
 - The list of J_{ij} values that make up the diagonal of the J_{ij} matrix. These are the values that govern the interaction between each pair of spins. It is created by the `generate_jij` function
- `hlist`
 - The list of h values over which the ground state probabilities will be found.

Returns:

- `probsoverh`
 - A dictionary where each key is an h value from the given `hlist`, and the value associated with each key is another dictionary where each key is the string of binary representing a ground state and the value is its respective probability for the given h value.
- `probs1, probs 2, probs3, probs4, ...`
 - The probabilities of each individual ground state over the range of h values.

7. `hamming_distance(length, Hintdiag)`
- This function finds the hamming distance between every combination of classical ground states.

Parameters:

- `length`
 - The number of spins in the system
- `Hintdiag`
 - The diagonal of the interaction term from the Hamiltonian. This is generated by the `sg_interaction` function.

Returns:

- N/A
 - This wont return a value but rather a series of print statements which tell you what the hamming distance is for every possible pair of ground states.
8. `max_variance_ratio(distancedict, hlist)`
 - This function calculates the coefficient of variance of entanglement distance for an instance over a range of h values. It will then find the h value which results in the largest coefficient of variance. This is generally indicative of interesting behavior in the system at the returned field strength. Additionally it calculates the standard deviation and mean of the set of entanglement distances over the same list of h values.

Parameters:

- `distancedict`
 - A dictionary of all the entanglement distances for every possible spin pair over a range of h values. The h value is the key, and the value is the list of distances for that field strength. For this function to work, the hlist input must be identical to the h values that make up the keys of this dictionary.
- `hlist`
 - The list of field strengths over which the distances have been calculated

Returns:

- `ratiodict`
 - A dictionary of the coefficient of variance for the calculated distances over the range of h values. The key is the h value and the value is the coefficient of variance at that field strength.
- `maxh`
 - The field strength at which the largest coefficient of variance for distance occurs.
- `max ratio`
 - The value of the maximum coefficient of variance for distance
- `stdevdict`
 - A dictionary of the standard deviation for the calculated distances over the range of h values. The key is the h value and the value is the standard deviation at that field strength.
- `meandict`
 - A dictionary of the mean of the calculated distances over the range of h values. The key is the h value and the value is the mean distance for that field strength.

9. `staggered_magnetization(length, states)`
– This function will calculate the staggered magnetization operator for a given instance. This is then used to ultimately find the phase transition. However, this function can only be used for degeneracy 2 instances.

Parameters:

- `length`
– The number of spins in the system
- `states`
– The list of classical ground states of the instance. These are represented as binary strings, where 0 is spin up and 1 is spin down. This list is generated by the function `find_degeneracy()`

Returns:

- `stagemag`
– The staggered magnetization operator for the given instance. In order to find the phase transition point, the expectation value of this operator must be plotted over a range of field strengths, and the transition point is the point of maximum slope.
10. `find_transitions(xaxis, stagemagvals, ratiodict, stdevdict, meandict)`
– This function will find the phase transitions for a given instance according to the staggered magnetization, coefficient of variance of distance, standard deviation of distance, and mean of distance. It does this by finding the maximum slope of the staggered magnetization and mean as well as the peak of the standard deviation and coefficient of variance over a range of field strengths. Note, that because it includes the staggered magnetization, this function can only be used for degeneracy 2 instances.

Parameters:

- `xaxis`
– The x axis of the plot for which this function finds the maximum slope and/or the peaks. By necessity, this is the list of field strengths over which the values have been found
- `stagemagvals`
– The list of expectation values of staggered magnetization over the set range of field strengths
- `ratiodict`
– The dictionary of coefficient of variance values which is returned by the function, `max_variance_ratio()`.

- `stdevdict`
 - The dictionary of standard deviation values which is returned by the function, `max_variance_ratio()`.
- `meandict`
 - The dictionary of mean values which is returned by the function, `max_variance_ratio()`.

Returns:

- `x_max_slope1`
 - The field strength corresponding to the point of maximum slope of the staggered magnetization curve. This corresponds to the phase transition point for the system.
- `peak_x2`
 - The field strength corresponding to the peak of the coefficient of variance curve. This is an approximation of the phase transition point.
- `x_max_slope2`
 - The field strength corresponding to the point of maximum slope of the mean curve. This is an approximation of the phase transition point.
- `peak_x1`
 - The field strength corresponding to the peak of the mean curve. This is an approximation of the phase transition point.

11. `mutual_info_fast(length, psi, A, B)`
 - This function finds the mutual information for a single pair of spins in a given system as well as the corresponding mutual information distance. This one calculates the reduced density matrices in a slightly different way, making it faster for larger systems.

Parameters:

- `length`
 - The number of spins in the system
- `psi`
 - The ground state of the system, given as a quantum object.
- `A`
 - The first spin of the pair. It must be passed into the function in the form `[A]`
- `B`
 - The second spin of the pair. It must be passed into the function in the form `[B]`

Returns:

- mutualinfo
 - The mutual information for the given pair of spins. The mutual information is not normalized.
- distance
 - The mutual information distance between the pair of spins. This distance is normalized by division of the maximum value $2 \log(2)$

3 Data Generation and Writing

1. save_dict_to_file(dictionary, filename)
 - Given a dictionary, this function creates a text file where each line corresponds to a key in the dictionary. Each line is formatted as Key: Value

Parameters:

- dictionary
 - The dictionary to write to a text file
 - filename
 - The desired name for the resulting file. It must be written as a string including the quotes around it.
2. save_dict_to_array_file(dictionary, filename)
 - Given a dictionary, this function creates a text file where each line corresponds to a key in the dictionary. Each line is formatted as an array where the rows are the key followed by the value

Parameters:

- dictionary
 - The dictionary to write to a text file
 - filename
 - The desired name for the resulting file. It must be written as a string including the quotes around it.
3. generate_sorted_classical_data(length, num_ones, seedlist)
 - This will generate the classical data for every instance given by the supplied length, and seeds in the given seed list

Parameters:

- length
 - The number of spins in the system

- `num_ones`
 - This is how many positive ones you would like in the ultimate list of `jij` values. Typically the list is half positive one and half negative one
- `seedlist`
 - The list of seeds that will generate the instances

Returns:

- `sorted_seeds`
 - This function returns a dictionary of sorted seeds. The dictionary contains the seed as the key and the value is a list of minimum classical energy, the classical degeneracy, and a list of the classical ground state. The seeds in the dictionary will be sorted from highest to lowest degeneracy.
4. `read_array_data(filename)`
- This will read from a text file created by the `save_dict_to_array()` function and return the information also as an array

Parameters:

- `filename`
 - The name of the file to read. It must be given inside quotes.

Returns:

- `array`
 - The array of data as seen in the text file.
5. `read_classical_data(filename)`
- This will read from a text file created by the `generate_sorted_classical_data()` function and return the information also as an array. This function is exclusive to reading files generated by this function which contain classical data.

Parameters:

- `filename`
 - The name of the file to read. It must be given inside quotes.

Returns:

- array
 - The array of data as seen in the text file.
- 6. `generate_mutual_information_data(length, num_ones, seedlist, Hfield, hval)`
 - This will generate lists of mutual information, as given by the `mutual_info()` function, over a list of seeds and return a dictionary where the key is the seed and the value is the list of mutual information

Parameters:

- `length`
 - The number of spins in the system
- `num_ones`
 - This is how many positive ones you would like in the ultimate list of `jij` values. Typically the list is half positive one and half negative one
- `seedlist`
 - The list of seeds that will generate the instances
- `Hfield`
 - The field term of the Hamiltonian for the specified length.
- `hval`
 - The field strength for each instance.

Returns:

- `seeddict`
 - The dictionary containing the mutual information for each seed. The key is the seed and the value is the list of mutual information as it is generated in the function `mutual_info()`
- 7. `generate_transitions_data(length, Hfield, seedlist, hvals)`
 - This will generate phase transition data over a range of `h` values for all of the instances dictated by the seed list provided.

Parameters:

- `length`
 - The number of spins in the system
- `Hfield`
 - The field term of the Hamiltonian for the specified length.
- `seedlist`
 - The list of seeds that will generate the instances
- `hvals`
 - The list of field strengths for each instance.

Returns:

- transitionsdict
 - The dictionary containing the phase transition point from the staggered magnetization and the various proxies: mean, coefficient of variance, and standard deviation. The key in the dictionary is the seed that generated the instance and the value is a list containing the transition points in the order listed above.
8. classical_data_by_line(length, num_ones, seedlist, filename)
- This will write to disk the classical data for every seed in the seed list. It will create a text file and append to it seed by seed.

Parameters:

- length
 - The number of spins in the system
- num_ones
 - This is how many positive ones you would like in the ultimate list of jij values. Typically the list is half positive one and half negative one
- seedlist
 - The list of seeds that will generate the instances
- filename
 - The name of the generated file. It must be given inside quotes.

Returns:

- N/A
 - It will write data to the text file for every seed rather than by taking in a dictionary with all the data and writing it all at once.
9. generate_mutual_info_by_line(length, num_ones, seedlist, Hfield, hval, filename)
- This will write to disk the mutual information data for every seed in the seed list for the given field strength. It will create a text file and append to it seed by seed.

Parameters:

- length
 - The number of spins in the system
- num_ones
 - This is how many positive ones you would like in the ultimate list of jij values. Typically the list is half positive one and half negative one

- seedlist
 - The list of seeds that will generate the instances
- Hfield
 - The field term of the Hamiltonian for the specified length.
- hval
 - The field strength for each instance.
- filename
 - The name of the generated file. It must be given inside quotes.

Returns:

- N/A
 - It will write data to the text file for every seed rather than by taking in a dictionary with all the data and writing it all at once.

4 Spectral Clustering

1. create_affinity_matrix(length, combinations, minfolist)
 - This will create the affinity matrix by rearranging the mutual information values. This affinity matrix is used to analyze clustering.

Parameters:

- length
 - The number of spins in the system
- combinations
 - Every possible combination of spins. It should be a list that goes through the combinations in the order of [0,1], [0,2], [0,3], and so on.
- minfolist
 - The list of mutual information for every combination of spins. This should be in the same order as the combinations list. This list is generated by either above mutual information function.

Returns:

- affmat
 - The function returns the affinity matrix as a numpy array.
2. affmat_analysis(affmat)
 - This will diagonalize the affinity matrix, and, through the maximum eigenvector gap, estimate the number of clusters in the system. It will also return the max gap as well as the normalized max gap and the second and third eigenvalue gaps.

Parameters:

- `affmat`
 - The affinity matrix as a numpy array. Taken from the function `create_affinity_matrix()`, or `rearrange_affmat()`

Returns:

- `num_clusters`
 - The estimated number of clusters in the system.
 - `max_gap`
 - The largest gap between eigenvalues of the affinity matrix
 - `max_gap_norm`
 - The largest gap between eigenvalues, normalized by the largest eigenvalue
 - `second_gap`
 - The second largest gap between eigenvalues of the affinity matrix
 - `third_gap`
 - The third largest gap between eigenvalues of the affinity matrix
3. `graph_conductance(affmat, clusters)`
 - This will calculate the conductance of the instance corresponding to the given affinity matrix. This function assumes two clusters.

Parameters:

- `affmat`
 - The affinity matrix as a numpy array. Taken from the function `create_affinity_matrix()`, or `rearrange_affmat()`
- `clusters`
 - The cluster object returned from sklearn's `SpectralClustering()` and `.fit_predict()` functions. It is a list of the same length as the system size, where the numbers in each index indicate which cluster that spin belongs to.

Returns:

- `conductance`
 - The calculated graph conductance for the given instance.
4. `rearrange_affmat(affmat, clusters)`
 - This will rearrange the affinity matrix to block diagonalize it, where each block on the diagonal corresponds to a cluster in the system. Note this only works for a system with two clusters.

Parameters:

- `affmat`
 - The affinity matrix as a numpy array. Taken from the function `create_affinity_matrix()`, or `rearrange_affmat()`
- `clusters`
 - The cluster object returned from sklearn's `SpectralClustering()` and `.fit_predict()` functions. It is a list of the same length as the system size, where the numbers in each index indicate which cluster that spin belongs to.

Returns:

- `new_order`
 - The rearranged order of columns and rows from the original affinity matrix
 - `rearranged_affmat`
 - The rearranged affinity matrix according to the new order. It will now be block diagonalized.
5. `eigenvector_overlap_rearranged(new_affmat, eigenvec_index)`
 - This will calculate the conductance of the instance corresponding to the given affinity matrix. This function assumes two clusters.

Parameters:

- `new_affmat`
 - The rearranged affinity matrix as a numpy array. This will only work with the rearranged affinity matrix. It is given by the function `rearrange_affmat()`.
- `eigenvec_index`
 - The index of the eigenvector for which you want to find the cluster overlaps.

Returns:

- `overlaps`
 - A list containing the given eigenvector's overlap with each cluster

5 Miscellaneous

1. `svd(a)`
 - Takes a given matrix and performs singular value decomposition on it. It returns the three resultant matrices from a singular value decomposition:

u , Σ , and v . Using the function will also print out the original matrix, eigenvalues, eigenvalues trimmed – setting sufficiently small values to zero, the eigenvectors, the singular values, u , Σ , v , as well as a multiplication of the three to test if the resulting matrix is the same as the original.

Parameters:

- a
 - The matrix upon which the function will perform singular value decomposition.

Returns:

- u
 - The first matrix of the singular value decomposition
 - Σ
 - The second matrix of the singular value decomposition
 - v
 - The first matrix of the singular value decomposition
2. `vec.to_matrix(length, vector, a_index, b_index)` – This will map a vector to a matrix only for a spin chain of size 3 what has been bipartitioned into sections a and b .

Parameters:

- `length`
 - The size of the spin system (for this function specifically, it will only work for a length of 3).
- `vector`
 - The vector to map to a matrix
- `a_index`
 - The positions of the spins in partition a
- `b_index`
 - The positions of the spins in partition b

Returns:

- `matrix`
 - The matrix mapping of the vector from the partitioned system